Information gain and backpropagation: worked solutions

Each problem starts with the full statement, then the solution step by step. Plain text in italics explains why each step is done; the formulas and numbers are what you write on the answer sheet.

Formula (printed on the practice sheets):

$$\text{IGain}(T) = I\left(\tfrac{p}{p+n}, \tfrac{n}{p+n}\right) - \sum_{k=1}^{K} \frac{p_k + n_k}{p+n}\, I\left(\tfrac{p_k}{p_k+n_k}, \tfrac{n_k}{p_k+n_k}\right), \qquad I(a,b) = -a\log_2 a - b\log_2 b$$

Calculator: $\log_2 x = \ln x / \ln 2$. Example: $\log_2 0.75$ is typed `ln(0.75)/ln(2)`.

Problem 1

Statement. We build a decision tree that predicts whether a sensor node fails during the day. Each situation is described by two attributes: A. Battery level: Low (L), Medium (M), High (H); B. Temperature: Hot (Ho), Cold (Co).

Ex.	Battery	Temp.	Failed?	Ex.	Battery	Temp.	Failed?
1	L	Ho	Y	7	L	Ho	N
2	M	Co	Y	8	M	Ho	Y
3	H	Ho	N	9	H	Ho	N
4	L	Co	Y	10	L	Co	Y
5	M	Ho	N	11	M	Co	N
6	H	Co	N	12	H	Co	N

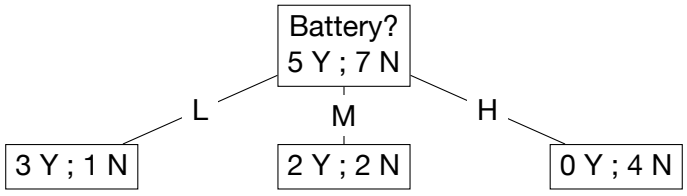
We test the battery level at the root node. Draw the decision tree resulting from this test and calculate the information gain of the test based on the 12 training samples.

Solution.

Step 1. Count the examples in each branch and draw the tree.

The test splits the 12 examples by battery level. Ignore the temperature column: it is not the attribute being tested. For each value, count how many Y and how many N fall in it.

L: examples 1, 4, 7, 10, answers Y, Y, N, Y, so 3 Y and 1 N.
M: examples 2, 5, 8, 11, answers Y, N, Y, N, so 2 Y and 2 N.
H: examples 3, 6, 9, 12, answers N, N, N, N, so 0 Y and 4 N.
Root: 3 + 2 + 0 = 5 Y and 1 + 2 + 4 = 7 N.



Step 2. Information needed at the root.

This measures how mixed the answers are before the test. Turn the counts into proportions first: 5 Y out of 12 is 5/12, 7 N out of 12 is 7/12. Never put the raw counts 5 and 7 in the formula.

$$I\left(\tfrac{5}{12}, \tfrac{7}{12}\right) = -\tfrac{5}{12}\log_2\tfrac{5}{12} - \tfrac{7}{12}\log_2\tfrac{7}{12} = 0.526 + 0.454 = 0.980 \text{ bit}$$

Check: with two classes the result is always between 0 and 1. A negative number or a number above 1 means the proportions are wrong.

Step 3. Information needed in each branch.

Same formula, but with the proportions inside the branch. Two shortcuts save time: a branch with only one class (pure) needs 0 bit; a branch split exactly half and half needs 1 bit.

L, 3 Y and 1 N out of 4:	$I(\tfrac{3}{4}, \tfrac{1}{4}) = -\tfrac{3}{4}\log_2\tfrac{3}{4} - \tfrac{1}{4}\log_2\tfrac{1}{4} = 0.311 + 0.5 = 0.811$
M, 2 Y and 2 N out of 4:	equal chances, so $I = 1$ (no calculation needed)
H, 0 Y and 4 N out of 4:	pure node, so $I = 0$ (no calculation needed)

Step 4. Remaining information: weight each branch by its share of the examples.

A branch with more examples counts more. Its weight is simply the number of examples in the branch divided by the total: here each branch has 4 of the 12 examples, so each weight is 4/12. The weight is a plain division, there is no logarithm in it.

$$\tfrac{4}{12}\cdot 0.811 + \tfrac{4}{12}\cdot 1 + \tfrac{4}{12}\cdot 0 = 0.270 + 0.333 + 0 = 0.604 \text{ bit}$$

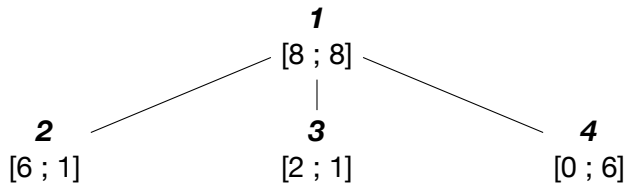
Step 5. Gain = information at the root minus remaining information.

The gain is how much the test reduced the uncertainty. 0.604 is not the answer: it is what is left after the test. Always finish with this subtraction.

$$\text{Gain} = 0.980 - 0.604 = \textbf{\textit{0.376 bit}}$$

Problem 2

Statement. We train the decision tree below for a binary decision. Every example belongs to class A or class B; there were 8 training examples of each class. The numbers in brackets show how many examples of class A and of class B reached each node (class A first). What is the information gain of the test performed at node 1? (Justify with a calculation.)



Solution. The tree is already drawn and the counts are given, so the work starts at the root.

Step 1. Root (node 1): 8 A and 8 B out of 16.

Half and half, so the shortcut applies.

$I(\tfrac{8}{16}, \tfrac{8}{16}) = 1$ bit.

Step 2. Each child node, with its own proportions.

Add the two numbers in the brackets to get the size of the node, then divide.

Node 2, [6 ; 1], 7 examples: $I(\frac{6}{7}, \frac{1}{7}) = -\frac{6}{7}\log_2\frac{6}{7} - \frac{1}{7}\log_2\frac{1}{7} = 0.191 + 0.401 = 0.592$

Node 3, [2 ; 1], 3 examples: $I(\frac{2}{3}, \frac{1}{3}) = -\frac{2}{3}\log_2\frac{2}{3} - \frac{1}{3}\log_2\frac{1}{3} = 0.390 + 0.528 = 0.918$

Node 4, [0 ; 6], 6 examples: pure node, so $I = 0$

Step 3. Remaining information.

Weights: 7, 3 and 6 examples out of 16.

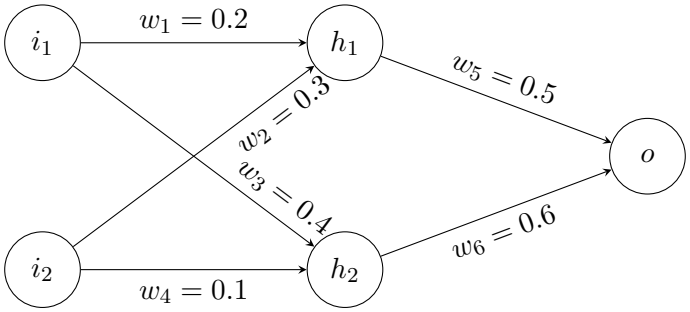
$$\frac{7}{16} \cdot 0.592 + \frac{3}{16} \cdot 0.918 + \frac{6}{16} \cdot 0 = 0.259 + 0.172 + 0 = 0.431 \text{ bit}$$

Step 4. Gain.

$$\text{Gain} = 1 - 0.431 = \boldsymbol{0.569 \text{ bit}}$$

Problem 3

Statement. The network below has 2 inputs, 2 hidden neurons and 1 output neuron. All neurons use the sigmoid activation $f(x) = 1/(1 + e^{-x})$. The hidden neurons share the bias $b_1 = 0.3$, the output neuron has the bias $b_2 = 0.2$. The error is $E = \frac{1}{2}(t - y)^2$. Input: $i_1 = 0.1$, $i_2 = 0.5$. Desired output (target): $t = 0.8$. Learning factor: $\eta = 0.5$.



a) Perform the forward pass: calculate the outputs of h_1, h_2 and o.
b) Calculate the value of the weight w_5 after the training step.
c) Calculate the value of the weight w_1 after the training step.

Solution.

a) Forward pass, one neuron at a time, from left to right.

Each neuron does two things. First it adds up its inputs, each multiplied by the weight of its arrow, and then adds its bias: this sum is called net. The bias is an extra number that belongs to the neuron, not to an arrow; the statement gives it ($b_1 = 0.3$ for h_1 and h_2, $b_2 = 0.2$ for o). Second, it puts net into the sigmoid to get its output y. The output of a hidden neuron then becomes an input of the output neuron.

Neuron h_1. Its arrows come from i_1 (weight w_1) and i_2 (weight w_2); its bias is b_1.

$$net_{h1} = i_1 w_1 + i_2 w_2 + b_1 = 0.1 \cdot 0.2 + 0.5 \cdot 0.3 + 0.3 = 0.02 + 0.15 + 0.3 = 0.47 \qquad y_{h1} = \frac{1}{1 + e^{-0.47}} = 0.615$$

Neuron h_2. Arrows from i_1 (w_3) and i_2 (w_4); bias b_1.

$$net_{h2} = i_1 w_3 + i_2 w_4 + b_1 = 0.1 \cdot 0.4 + 0.5 \cdot 0.1 + 0.3 = 0.04 + 0.05 + 0.3 = 0.39 \qquad y_{h2} = \frac{1}{1 + e^{-0.39}} = 0.596$$

Neuron o. Its inputs are the outputs of h_1 and h_2, through w_5 and w_6; bias b_2.

$$net_o = y_{h1} w_5 + y_{h2} w_6 + b_2 = 0.615 \cdot 0.5 + 0.596 \cdot 0.6 + 0.2 = 0.308 + 0.358 + 0.2 = 0.865 \qquad y_o = \frac{1}{1 + e^{-0.865}} = 0.704$$

The network answers 0.704 while the target is 0.8: the output is too low. Steps b and c correct the weights so that the next answer gets closer to 0.8.

b) New value of w_5.

Step b1. Output delta. The output delta measures the error of the output neuron, multiplied by the slope of the sigmoid at that point. The slope of the sigmoid is $y(1-y)$.

$$\delta_o = (y_o - t) \cdot y_o(1 - y_o) = (0.704 - 0.8) \cdot 0.704 \cdot (1 - 0.704) = (-0.096) \cdot 0.704 \cdot 0.296 = -0.0201$$

The output is below the target, so δ_o is negative, and the weights will go up.

Step b2. Find the input of w_5. Look at the arrow of w_5: it starts at h_1 and ends at o. What travels along it is the output of h_1.

Input of w_5 = $y_{h1} = 0.615$.

Step b3. Update. Rule: new weight = old weight − learning factor × delta × input.

$$w_5^{new} = w_5 - \eta \cdot \delta_o \cdot y_{h1} = 0.5 - 0.5 \cdot (-0.0201) \cdot 0.615 = 0.5 + 0.0062 = \mathbf{0.5062}$$

c) New value of w_1.

Step c1. Hidden delta of h_1. w_1 does not touch the output directly: it reaches it through h_1. So the error has to be carried back to h_1. Take the output delta, multiply by the weight that connects h_1 to the output (the old $w_5 = 0.5$, not the new one), then by the slope of the sigmoid at h_1, $y_{h1}(1 - y_{h1})$.

$$\delta_{h1} = \delta_o \cdot w_5 \cdot y_{h1}(1 - y_{h1}) = (-0.0201) \cdot 0.5 \cdot 0.615 \cdot (1 - 0.615) = (-0.0201) \cdot 0.5 \cdot 0.615 \cdot 0.385 = -0.00238$$

Step c2. Find the input of w_1. The arrow of w_1 starts at i_1, so its input is i_1.

Input of w_1 = $i_1 = 0.1$.

Step c3. Update, same rule as in b3.

$$w_1^{new} = w_1 - \eta \cdot \delta_{h1} \cdot i_1 = 0.2 - 0.5 \cdot (-0.00238) \cdot 0.1 = 0.2 + 0.00012 = \mathbf{0.20012}$$

Order to remember

1. Forward: for each neuron, net = $\sum$ (input × weight) + bias, then y = sigmoid(net).
2. Output delta: $\delta_o = (y_o - t)\, y_o(1 - y_o)$.
3. Output weights: new $w = w - \eta\, \delta_o \times$ (output of the hidden neuron the arrow starts from).
4. Hidden delta: $\delta_h = \delta_o \times w_{out}^{old} \times y_h(1 - y_h)$.
5. Hidden weights: new $w = w - \eta\, \delta_h \times$ (input the arrow starts from).

Sign check: output below the target gives a negative delta, and every weight increases.